SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- EXPERIMENTS

Course Code:	**CSA12**	**Course Title:**	**Computer Architecture**
CO Assessed:	**CO3**	**Assessment:**	**ASSESSMENT TOOL 1- EXPERIMENTS**
Weightage:	**40%**	**Total Marks:**	**25**
CO3 Statement:	*Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT). (BL6)*		
Student Name:	**Shafina. A**	**Reg.No.:**	**192521474**
Department:	B. Tech IT	**Date:**	

ANNEXURE A

1. Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.

Answer:

Objective:

To simulate register transfer operations and basic ALU operations using multiple registers.

Registers Used:

- R1 = 10
- R2 = 5
- R3 = Output Register

Operations Performed:

Operation	Res
R3 ← R1 + R2	15
R3 ← R1 − R2	5

R3 ← R1 AND R2 0

R3 ← R1 OR R2 15

Sequence of Operations:

1. Load data into R1 and R2.
2. Transfer operands to the ALU.
3. ALU performs the selected operation.
4. Store the result in R3.

Role of the ALU:

- Performs arithmetic operations (addition and subtraction).
- Performs logical operations (AND and OR).
- Sends the computed result back to the destination register.

Conclusion:
The simulation demonstrates efficient register-to-register data transfer, with the ALU processing arithmetic and logical operations before storing the result.

2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyze how multiple bus structures improve parallel data transfer and reduce bottlenecks.

Answer:

Objective:
To compare single-bus and multi-bus architectures.

Single-Bus Organization

- One bus is shared by all components.
- Only one data transfer occurs at a time.
- Simpler design but slower performance.

Multi-Bus Organization

- Multiple buses allow simultaneous data transfers.
- Registers, memory, and I/O can communicate in parallel.
- Reduces waiting time and increases throughput.

Feature	Single Bu	Multi Bus

Data Transfer	One at a time	Multiple simultaneously
Speed	Lower	Higher
Hardware Cost	Low	High
Performance	Moderate	Better

Conclusion:
A multi-bus organization provides faster data transfer and reduces bus contention, making it more efficient than a single-bus organization.

3. Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.

Answer:

Objective:
To study the performance of pipelined instruction execution.

Pipeline Stages

1. Instruction Fetch (IF)
2. Instruction Decode (ID)
3. Execute (EX)
4. Memory Access (MEM)
5. Write Back (WB)

Example Instructions

- ADD R1, R2, R3
- SUB R4, R1, R5
- AND R6, R4, R2
- OR R7, R6, R1

Performance Comparison

Parameter	Non-Pipelin	Pipelin
Clock Cycles (4 Instructions)	20	8
Throughput	Low	High
Speedup	1×	≈2.5×

Conclusion:
Instruction pipelining improves processor performance by executing multiple instructions simultaneously, resulting in higher throughput and reduced execution time.

4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.

Answer:

Objective:
To study pipeline hazards and their solutions.

Data Hazard

Occurs when an instruction depends on the result of a previous instruction.

Solution: Data Forwarding or Stalling.

Control Hazard

Occurs due to branch instructions changing program flow.

Solution: Branch Prediction or Pipeline Flush.

Structural Hazard

Occurs when two instructions require the same hardware resource simultaneously.

Solution: Separate hardware resources or pipeline scheduling.

Hazard	Cause	Solution
Data Hazard	Operand dependency	Forwarding, Stall
Control Hazard	Branch instruction	Branch Prediction
Structural Hazard	Resource conflict	Additional Hardware

Conclusion:
Using forwarding, branch prediction, and proper resource allocation significantly improves pipeline efficiency and reduces execution delays.

5. Analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

Answer:

Objective:
To analyze advanced processor execution techniques.

Superscalar Execution

- Executes multiple instructions in one clock cycle using multiple execution units.

Out-of-Order Execution

- Executes independent instructions before earlier stalled instructions.
- Improves CPU utilization.

Speculative Execution

- Predicts future instructions and executes them before confirmation.
- Reduces branch delays.

Hyper-Threading

- Executes two logical threads on one physical core.
- Improves processor utilization.

Simultaneous Multithreading (SMT)

- Executes instructions from multiple threads simultaneously.
- Maximizes execution unit usage.

Technique	Benefit
Superscalar	Multiple instructions per cycle
Out-of-Order	Reduces idle CPU time
Speculative Execution	Faster branch execution
Hyper-Threading	Better CPU utilization
SMT	Higher instruction throughput

Conclusion:

Advanced execution techniques improve instruction throughput, reduce processor idle time, and enhance overall CPU performance by executing multiple instructions and threads in parallel.

Code of Conduct:

*I __Shafina. A (*192521474*)____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: *Shafina. A*

MARKS ALLOCATION

	Understanding of Concepts (20%)	Experimental Design (20%)	Use of Tools (25%)	Analysis of Results (20%)	Documentation (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation